General notes

Permanent Faults

- ATPG (Automated Test Pattern Generation) & BIST
- Test Configurations
    - Using specific bitstreams to cover all cases
- External testing
    - Automated Test Equipment (ATE) checks for continuity, shorts, and parametric failures during final production

Transient/Soft Errors

- Fault injection (Software/Simulation)
    - Injecting bitflips into the HDL configuration
- Physical Fault Injection
    - Uses particle beams or lasers to induce errors for radiation testing.
- Scrubbing
    - Reads and corrects configuration memory (SRAM or BRAM) to prevent error accumulation

Testing

- Built In Self Test (BIST)
    - Embedded testing logic that is useful for internal connections with minimal overhead.
- Independent Functional Testing (IFT)
    - Uses customized bitstreams to find stuck at faults in underlying logic and interconnections often using the DONE pin
- Embedded Logic Analyzers
    - Xilinx, ChipScope used for in circuit debugging and fault observation

Fault Tolerance and Recovery

- Hardware Redundancy
    - Swapping faulty resources with spares
- Reconfiguration
    - Rerouting logic to bypass faulty elements
- Dynamic Partial Reconfiguration
    - Reconfiguring only parts of the design at runtime to work around errors

Obj

- Show how faults have visual impacts and effects on specialized hardware

- Moa
    - Fault injection
    - Real time logic analyzers
- Expected results
    - Expect things based off of rom table and purp make a fault and check to see what happens
    - Built in self test

Horizontal timing

- Reason for 4h and 8h
    - Tank Battalion is built around a fixed monitor timing standard for its display hardware. The horizontal counter is arranged so that certain bit patterns naturally line up with:
        - end of visible video
        - front porch
        - HSYNC pulse
        - back porch
        - start of visible video

Vertical timing

5A takes the base horizontal clock and produces clean divide-by-16/32/64/256 timing bits so the rest of the board can (a) know where it is in the scan line and (b) build blanking/sync/vertical clocks with very little extra logic.

Dip switch

The CPU writes a value onto **DBQ** (data bit).

The CPU chooses an address (AB0–AB2).

A write strobe (**OUT1 / OUT0 / WE**) selects which latch chip is being written.

That latch stores the bit onto one of its Q outputs.

Those Q outputs are literally the named control lines: **HIT, SHOOT, HI/LOW, OFF/ON, S1, S2**, etc.

Those control lines then open 4066 switches, trigger envelopes, change oscillator rates, etc. in the analog sound sections you were analyzing.

https://drive.google.com/file/d/12TywABaQdk6__2lIMciBZsfHLqNTfur7/view?usp=sharing

Sources

https://www.eng.auburn.edu/~agrawvd/COURSE/E7250_06/TERM/Qin_ADFPGAT.pdf
https://www.youtube.com/watch?v=qxgM9NPrhqw&list=PLaPDcLu5g01k3_iUxi79yFyeGkS7iFRXz
https://github.com/nic24-rgb/TankBatt

Notes
https://web.goodnotes.com/s/tt507ooyQJOpE2gZjSJLIA

Power point
https://docs.google.com/presentation/d/1eq4YY7zGw6ZvAMZ1Qnv1v2-eT7ULdWpzIp9GHsjo2_8/edit?usp=sharing

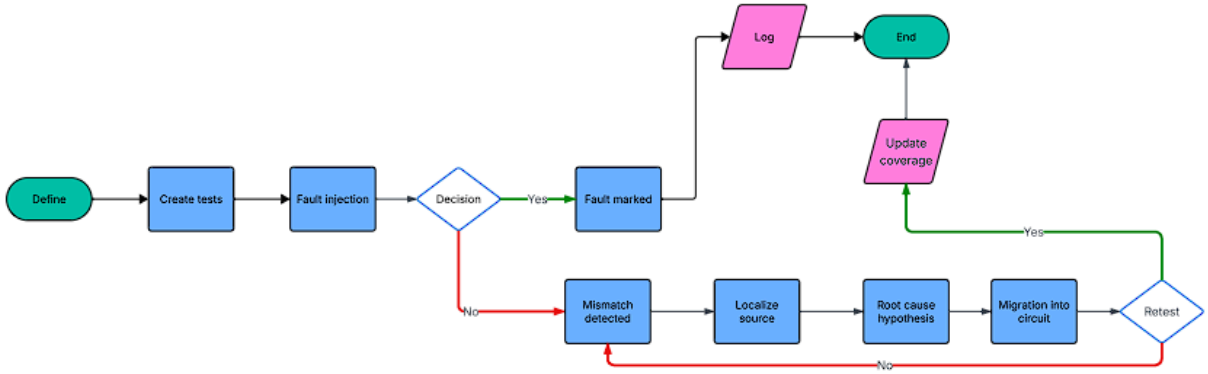
Project Proposal

Fault Simulation For Specialized Hardware

Kyle Osborn, Justin Spenker
Email: kosborn@ucdavis.edu, jspenker@ucdavis.edu

Introduction

This project focuses on the testing and validation of specialized hardware specifically targeting an FPGA based implementation of the arcade game Tank Battalion. The goal of this is to study how physical faults manifest in FPGA designs and how we can simulate, debug, and fault injection techniques which can be used to identify and trace issues. By analyzing the resulting behavior and tracing faults back to their architectural origins, this project will demonstrate the practical methods for ensuring functionality, reliability, and internal correctness.



Objectives

Our main objective is to study the testing of specialized hardware. In this case, an FPGA implementation of the arcade game Tank Battalion [1] will be used as a starting point. Our objective has multiple parts covering a variety of methods to test an FPGA design. First verification of the functionality in simulation by comparing component documentation to simulated results will be covered. Next, realtime analysis of the programmed FPGA using a logic analyzer to confirm the results match the simulation will be done. Finally, fault injection will be performed targeting specific parts of the arcade board's architecture. The tilemap generator, sprite renderer, video sync generator and more will each have SSL and SEU faults injected, with

the visual results analyzed. The faults will then be traced back to their origin from the perspective of debugging a fault of unknown origin using knowledge of the board's architecture. We expect to find that these methods thoroughly test an FPGA design to ensure its functionality and internal logic.

Time Schedule

Proposed Activities	Planned dates
Project definition and overview	Week 3 (Jan 18 - 24)
Survey FPGA and circuit diagram	Week 4 (Jan 25 - 31)
Fault modeling techniques and injection methods	Week 5 (Feb 1 - 7)
Analysis of simulation and debugging tools FPGA	Week 6 (Feb 8 - 14)
Implementation of faults in circuit	Week 7 (Feb 15 - 21)
Testing and debugging under injected fault conditions (Start first draft)	Week 8 (Feb 22 - 28)
Evaluation of fault coverage and detection effectiveness	Week 9 (March 1 - 7)
Documentation and final report	Week 10 (March 8 - 14)

References

[1] nic24-rgb, "TankBatt," *GitHub repository*. [Online]. Available: https://github.com/nic24-rgb/TankBatt. Accessed: Jan. 24, 2026.

[2] J. Qin, "A brief introduction to application-dependent FPGA testing," Dept. of Electrical and Computer Engineering, Auburn University, Auburn, AL, USA. [Online]. Available: https://www.eng.auburn.edu/~agrawvd/COURSE/E7250_06/TERM/Qin_ADFPGAT.pdf. Accessed: Jan. 24, 2026.

Progress Report

Progress Report For Fault Simulation For Specialized Hardware

Kyle Osborn, Justin Spenker
Email: kosborn@ucdavis.edu, jspenker@ucdavis.edu

Objective

Our main goal for this stage was getting an understanding of how the game operated on a hardware level and establishing a working FPGA instance of the game to support fault injections and repeatable experiments.

Work Completed

Justin

Reversed engineered and developed an understanding of the Tank Battalion PCB schematic. This involved breaking down the design into several functional subsystems and tracing signal flow.

- Analyzed horizontal and vertical timing (HSYNC, HBLANK, VBLANK, COMPOSYNC)
- Sound generation
- Control logic

Kyle

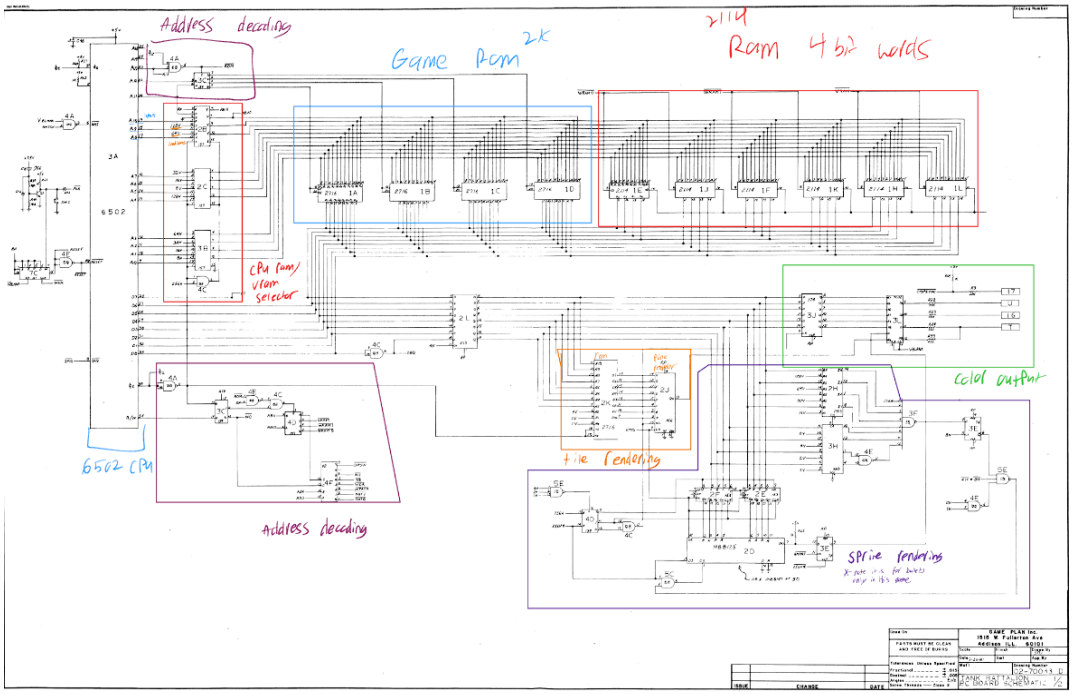
In parallel worked on bringing up a stand alone FPGA instance of Tank Battalion on the DE10-Nano on the open source MiSTer platform. This involved configuring the MiSTer environment, validating loading ROM and successfully running the game core. This setup provides a controlled environment for upcoming fault injections and experiments.

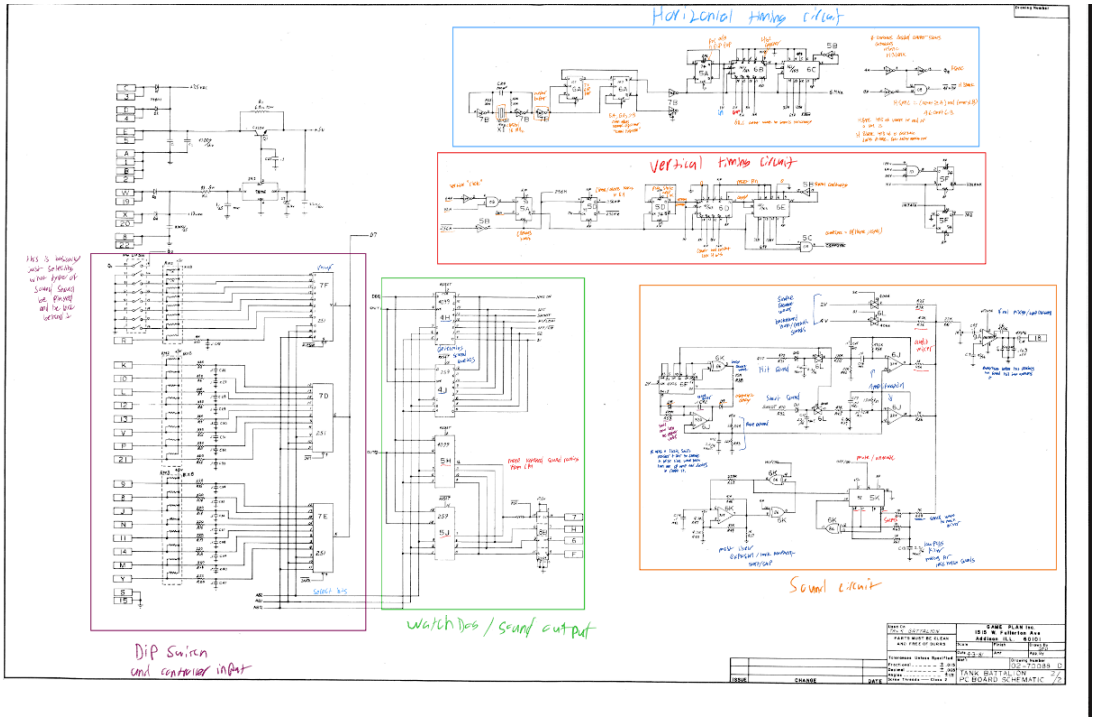
Next Steps

Our next objectives are to select targeted fault injection sites and apply fault models. We will then run a controlled experiment, log observations and compare results against predicted behavior from our schematic based understanding.

Summary

We established an FPGA sandbox (MiSTer) for controlled fault injection and began mapping candidate fault points from the original schematic.





Tab 5

Yes — that structure is solid, and it fits what you've already done. It also reads like a real engineering report: **methods → implementation example → predicted outcomes → observed outcomes → takeaways**.

A clean way to frame it is:

Paper concept

"Fault-testing methods for digital/embedded hardware, demonstrated on a real arcade PCB implemented on FPGA (Tank Battalion)."

Why this works

- You don't need to claim you "built a new system."
- Your schematic work becomes the justification for *why* certain fault sites matter.
- The MiSTer bring-up becomes the experimental platform.
- You can publish a clear mapping: **fault → symptom**, plus what test reveals it.

Recommended paper structure

1) Background

- What Tank Battalion hardware looks like at a systems level (CPU + timing + video + sound + I/O)
- Why timing logic is "critical" (sync/blanking) and why control latches matter

2) Fault models

Keep it small but standard:

- **Stuck-at-0 / stuck-at-1**
- **Transient bit flip (SEU-like)**

- **Clock glitch / timing skew**
- (Optional) **Burst faults** (multi-bit upset)

3) Test methods

Pick 2–3 “methods,” not 10:

- **Directed tests** (purposeful scenarios / “test vectors”): e.g., force many sprites, trigger shoot/hit repeatedly, move continuously, etc.
- **Runtime invariants / monitors**: e.g., HSYNC frequency must be ~15.7 kHz, VSYNC ~60 Hz, blanking must occur; watchdog resets if CPU stalls.
- **A/B comparison logging**: baseline vs faulted run, with a simple scoring rubric (lock lost, corruption, audio stuck, hang)

4) Tank Battalion case study

This is where you drop your “fault site map”:

- Horizontal counter bits / HSYNC decode
- Vertical counter bits / VBLANK decode
- Sound latch bits (HIT/SHOOT/HI-LOW/OFF-ON)
- ROM load path / checksum
- Reset/NMI/watchdog line

5) Expected outputs

Make a table like:

- **Fault site** | **Fault type** | **Expected symptom** | **How we detect it**

Examples:

- HSYNC stuck low → no horizontal lock → display collapse/black; detect by missing HSYNC edges
- VBLANK stuck high → blank screen; detect by constant blanking
- Sound latch stuck high on SHOOT → continuous shot sound; detect by audio stuck state
- ROM bit flip in code region → deterministic crash at same gameplay step; detect by repeatability

6) Results

Even if results are limited, you can still show:

- which predictions matched
- which didn't (and why)
- what faults are "benign" vs "catastrophic"

7) Discussion

- Which subsystems are most sensitive
- What this implies for "vital equipment" class systems (timing, reset strategy, redundancy, monitoring)

8) Conclusion

- Your framework + what you learned

What to be careful about

Don't make it "a survey of every fault testing method ever." Keep it:

- **2–3 methods**

- **6–10 fault sites**
- **clear predicted vs observed behavior**

That's enough to be strong.

What you need to make it convincing (minimum)

Even if you're short on time, aim for:

- 1 figure: annotated schematic / subsystem diagram (you already have this)
- 1 table: fault site → expected symptom
- 1 table or chart: observed outcomes summary
- 2–3 screenshots or short logs showing before/after fault

If you want, I can outline a **1–2 page "Methods" section** and a **fault-symptom table** tailored specifically to Tank Battalion (timing + sound + latch control), so you can drop it straight into your paper.

Report

Background

System Overview

The Tank Battalion platform represents a typical early-1980s arcade hardware architecture. The system is composed of a 6502-based CPU subsystem, discrete horizontal and vertical video timing logic, memory-mapped input/output circuitry, and an analog sound path controlled by digital latch signals.

Unlike modern graphics systems that rely on framebuffers and GPUs, video output in Tank Battalion is generated through dedicated hardware timing circuits. Horizontal and vertical counters produce synchronization (HSYNC/VSYNC) and blanking (HBLANK/VBLANK) signals that directly control display stability and visible rendering. Game graphics are formed by character and sprite ROMs whose outputs are synchronized to this timing chain.

Audio is produced through a hybrid architecture: digital control lines enable oscillators, noise generators, and envelope-shaping stages implemented with discrete logic and operational amplifiers. As a result, internal digital signals are directly reflected in observable system behavior, including display stability, sprite rendering, and audio output. This tight coupling between logic-level signals and external outputs makes the system well suited for fault-injection analysis.

Observable Fault Predictions

Due to the deterministic and hardware-driven nature of the design, faults in critical subsystems produce immediate and measurable external effects. The most sensitive subsystems are the video timing chain and graphics generation path.

Video Timing Faults

Faults affecting horizontal or vertical synchronization logic typically produce the most immediately observable effects.

Horizontal Timing Faults

- **HSYNC stuck low/high or missing pulses:** Monitor loses horizontal lock; image tearing, sideways rolling, or complete loss of picture may occur.

- **HBLANK stuck asserted:** Display remains fully black due to continuous blanking.
- **HBLANK stuck deasserted:** Retrace lines or bright diagonal artifacts become visible.

Vertical Timing Faults

- **VSYNC missing or unstable:** Vertical rolling or frame drift.
- **VBLANK stuck asserted:** Black screen or partial blanked region.
- **COMPOSYNC incorrect polarity or pulse shape:** Complete loss of display synchronization.

Because synchronization signals directly control beam positioning and blanking intervals, even single-bit faults in counter or decode logic can produce catastrophic display instability.

Graphics ROM and Sprite Path Faults

Faults within the character/tile ROM path or sprite generation logic result in visible but non-catastrophic corruption. Since graphics data is fetched directly from ROM and shifted into pixel-generation hardware, any corruption in address lines or data bits can produce:

- Scrambled background tiles
- Corrupted fonts or terrain graphics
- Incorrect sprite shapes or sizes
- Displaced sprite positions
- Missing or partially rendered objects

Unlike timing faults, these errors typically preserve synchronization but alter image content. This distinction makes them useful for categorizing fault severity and subsystem sensitivity.

1) Why FPGAs fail in ways that look like software.

In FPGA systems, the same symptom can come from totally different root causes, so debugging has to be evidence based where it follows a workflow of observe -> measure -> isolate -> confirm

Script ish

Part 1: Background, Scope, and Tank Battalion Context

Today we are presenting FPGA faults using a physical, real system example, with the goal of understanding what failure looks like when the problem is not “just code,” but a physical issue in the FPGA board or its hardware implementation. In this talk, we will focus on three things: what the symptoms look like, how engineers diagnose the root cause, and what theoretical fault mechanisms we expect to produce those symptoms. To keep the discussion grounded, we will use the 1980’s retro arcade game Tank Battalion as the reference system. Instead of treating faults as abstract concepts, Tank Battalion gives us a running, observable system where failures show up clearly through video behavior, timing stability, and game logic outputs.

FPGA systems are unique because the “software” you write is ultimately compiled into physical hardware. Once the design is synthesized, placed and then routed, your behavior depends not only on the HDL logic, but also on timing margins, routing delay, clock stability, power integrity, and the state of configuration memory. That means a failure that looks like a software bug—random freezes, intermittent glitches, incorrect outputs—can actually be caused by timing violations, unstable clocking, routing sensitivity, or other board-level conditions. In other words, the same symptom can be created by multiple root causes, which is why FPGA debugging requires an evidence-first workflow rather than trial-and-error guessing.

The scope of this project is not to claim that Tank Battalion is “broken” or that every issue is a rare hardware defect. The scope is to show how engineers think when something fails in an FPGA-based system. We can first start with the symptoms and how we identify what internal signals and paths are involved. Then we run targeted tests to isolate the category of fault, and we compare the results to what the theory predicts. The goal is that a reader or viewer could take the same workflow and apply it to their own FPGA project, whether they are debugging a video pipeline, a controller state machine, or a data path that only fails under certain conditions.

Part 2: Understanding

Now the very first step we should take is understanding what we are getting into and building a baseline mental model of the system. Before you try to “fix” anything, you need a general picture of how data and control signals move through the board. If you don’t understand the data path, every symptom just looks like noise. Once you do understand it, symptoms start to narrow down into likely categories, and your testing becomes focused instead of random.

There are two main ways to build that picture. The first method is to pull up the schematic and work top-down. This is usually the fastest and cleanest approach because the schematic tells you what each section is supposed to do. You can identify the major chips, power rails, clocks, memory devices, connectors, and interfaces, and then you can start separating the board into functional blocks. For example, you can isolate the video path from the logic/control path, separate the power regulation from the digital logic, and trace how the FPGA connects to memory and external I/O. Once you do that, you can make simple “if this fails, what breaks” predictions, which becomes the foundation of your diagnostic workflow.

The second method is more hands-on: you can do the same system mapping using the physical board, following traces, checking component placement, and confirming how signals and power are routed in reality. This approach is slower, but it can be extremely valuable because it reveals what the schematic doesn’t always capture in practice—things like connector wear, cracked solder joints, corrosion, or subtle damage near high-stress areas. It also forces you to confirm how the board is actually built, not just how it was designed.

In most cases, the schematic-based approach is the easiest starting point, but physical inspection should not be skipped. A quick visual check can catch obvious problems early: discoloration from overheating, blown or bulging components, lifted pads, broken traces, or anything that simply looks “off” compared to the rest of the board. Even if nothing looks damaged, physical inspection still helps you identify which areas likely experience the most stress, which matters when you later start forming hypotheses about failure mechanisms.

The goal of this first step is simple: turn the board from a mystery into a set of subsystems. Once you can describe the board as “power → clock → FPGA → memory → output,” you can start linking symptoms to sections of the system. That’s when diagnosis becomes engineering: you are no longer guessing, you are narrowing possibilities based on how the data actually flows.

Part 3: Issues

Forcing VBLANK low did not disable the full display. Instead, the image remained largely intact, with visible red artifacts along the lower borders of text and textures. This suggests that in the implemented video pipeline, VBLANK is used primarily to control end-of-frame blanking behavior rather than to globally gate all active video. The observed artifact is consistent with incorrect vertical blank boundary handling and color data leakage near the bottom of the frame.

Tab 7

Best fault models to include

For your implementation, I would definitely include:

- **stuck-at-0 / stuck-at-1**
  This is the standard and should be your baseline.
- **open / missing pulse / clock loss**
  Very relevant for timing circuits.
- **delay faults**
  Especially useful on sync and blanking edges.
- **intermittent faults**
  Good for realism, but probably after the deterministic faults work.
- **bridging / short faults**
  Good if you want more realism, especially for adjacent bus or logic lines.

If you need to keep scope controlled, start with:

1. stuck-at faults
2. missing pulse faults
3. delay faults

That is already a strong implementation.

Phase 1

Implement fault injection on:

- HSYNC
- HBLANK
- horizontal counter reset/carry

Measure:

- monitor lock
- visible behavior
- waveform changes

Phase 2

Add:

- VSYNC
- VBLANK
- vertical counter reset/carry

Measure:

- frame roll
- blank screen
- frame collapse

Phase 3

Add:

- display-enable and video-address faults

Measure:

- missing/corrupted objects
- repeated graphics
- partial rendering failures

Phase 4

- input faults
- audio trigger faults
- intermittent fault injection

stuck right
vblank stuck 0- red underline

change tank color
change tank sprite

jump to game over
change score/lives

failing self test
messing with sound
mess with sprite rendering

Video topics

- Introduction
  - Background
    - Game
    - Circuit diagram
    - What we are going to be testing
- Body
  - Show demonstration of what we are doing
    - Show examples of obvious faults
    - Showing stuck at or bit flips
    - Color changes
    - Weird glitches
- Conclusion
  - Compare theoretical vs experimental
  - Did we expect these results?

https://docs.google.com/presentation/d/1eq4YY7zGw6ZvAMZ1Qnv1v2-eT7ULdWpzIp9GHsjo2_8/edit?usp=sharing

Script ish

Script ish

Part 1: Background, Scope, and Tank Battalion Context

Today we are presenting FPGA faults using a physical, real system example, with the goal of understanding what failure looks like when the problem is not “just code,” but a physical issue in the FPGA board or its hardware implementation. In this talk, we will focus on three things: what the symptoms look like, how engineers diagnose the root cause, and what theoretical fault mechanisms we expect to produce those symptoms. To keep the discussion grounded, we will use the 1980’s retro arcade game Tank Battalion as the reference system. Instead of treating faults as abstract concepts, Tank Battalion gives us a running, observable system where failures show up clearly through video behavior, timing stability, and game logic outputs.

FPGA systems are unique because the “software” you write is ultimately compiled into physical hardware. Once the design is synthesized, placed and then routed, your behavior depends not only on the HDL logic, but also on timing margins, routing delay, clock stability, power integrity, and the state of configuration memory. That means a failure that looks like a software bug—random freezes, intermittent glitches, incorrect outputs—can actually be caused by timing violations, unstable clocking, routing sensitivity, or other board-level conditions. In other words, the same symptom can be created by multiple root causes, which is why FPGA debugging requires an evidence-first workflow rather than trial-and-error guessing.

The scope of this project is not to claim that Tank Battalion is “broken” or that every issue is a rare hardware defect. The scope is to show how engineers think when something fails in an FPGA-based system. We can first start with the symptoms and how we identify what internal signals and paths are involved. Then we run targeted tests to isolate the category of fault, and we compare the results to what the theory predicts. The goal is that a reader or viewer could take the same workflow and apply it to their own FPGA project, whether they are debugging a video pipeline, a controller state machine, or a data path that only fails under certain conditions.

Part 2: Understanding

Now the very first step we should take is understanding what we are getting into and building a baseline mental model of the system. Before you try to “fix” anything, you need a general picture of how data and control signals move through the board. If you don’t understand the data path, every symptom just looks like noise. Once you do understand it, symptoms start to narrow down into likely categories, and your testing becomes focused instead of random.

There are two main ways to build that picture. The first method is to pull up the schematic and work top-down. This is usually the fastest and cleanest approach because the schematic tells you what each section is supposed to do. You can identify the major chips, power rails, clocks, memory devices, connectors, and interfaces, and then you can start separating the board into functional blocks. For example, you can isolate the video path from the logic/control path,

separate the power regulation from the digital logic, and trace how the FPGA connects to memory and external I/O. Once you do that, you can make simple “if this fails, what breaks” predictions, which becomes the foundation of your diagnostic workflow.

The second method is more hands-on: you can do the same system mapping using the physical board, following traces, checking component placement, and confirming how signals and power are routed in reality. This approach is slower, but it can be extremely valuable because it reveals what the schematic doesn’t always capture in practice—things like connector wear, cracked solder joints, corrosion, or subtle damage near high-stress areas. It also forces you to confirm how the board is actually built, not just how it was designed.

In most cases, the schematic-based approach is the easiest starting point, but physical inspection should not be skipped. A quick visual check can catch obvious problems early: discoloration from overheating, blown or bulging components, lifted pads, broken traces, or anything that simply looks “off” compared to the rest of the board. Even if nothing looks damaged, physical inspection still helps you identify which areas likely experience the most stress, which matters when you later start forming hypotheses about failure mechanisms.

The goal of this first step is simple: turn the board from a mystery into a set of subsystems. Once you can describe the board as “power → clock → FPGA → memory → output,” you can start linking symptoms to sections of the system. That’s when diagnosis becomes engineering: you are no longer guessing, you are narrowing possibilities based on how the data actually flows.

Part 3: Issues
This is where I think we should dive deeper into common faults and what we should expect from these issues. We should talk about fault injection methods, diagnostic tools, and where in the circuit we should look.

Today we are presenting FPGA faults using a real system example. Our goal is to understand what hardware failure looks like when the problem isn’t just software, but something physical happening inside an FPGA implementation.

The goal of this project is not to claim the system is broken, but to demonstrate how engineers approach debugging when something fails in an FPGA system.

We start by observing symptoms, identify the signals that might be responsible, run targeted tests, and compare the results with theoretical fault predictions.

FPGA systems are unique because the code we write eventually becomes physical hardware. After synthesis and routing, the behavior depends not only on HDL logic but also timing margins, routing delays, clock stability, and configuration memory.

Because of that, a failure that looks like a normal software bug—like a freeze or glitch—can actually be caused by physical timing or hardware issues. That means engineers must diagnose problems using evidence rather than guesswork.

To make this analysis concrete we used the 1980s arcade game Tank Battalion. This system is useful because its hardware architecture is simple and deterministic.

The video system is driven entirely by timing counters and ROM data, which means faults in the hardware produce immediate visual effects such as corrupted graphics or display instability.

This diagram shows the major subsystems in the Tank Battalion hardware architecture. Player inputs are processed by the 6502 CPU, which handles the game logic and interacts with ROM and RAM for program and data storage. The video timing logic generates synchronization signals like HSYNC and VSYNC that control the display scanning. Graphics and sprite generation hardware produces the final video output that is sent to the CRT display and audio devices.

The first step when debugging any hardware system is building a mental model of how it works. If you don't understand how signals move through the design, every symptom just looks random. Once you understand the architecture, symptoms start to narrow down into specific subsystems, which makes debugging much more focused. There are two main ways engineers build this understanding: schematic analysis and physical inspection.

Looking at the schematic lets us break the board into functional subsystems. We can isolate the video path from the control logic, trace how signals propagate, and predict what failures might occur if specific signals stop working correctly

The second approach is physical inspection. This step can reveal issues that schematics don't show, such as damaged connectors, cracked solder joints, or overheated components. Even if no damage is visible, it helps identify areas of the board that experience mechanical or electrical stress.

This project focuses on injecting faults into critical signals in the system. These include synchronization signals like HSYNC and VBLANK, as well as graphics and sprite logic. By intentionally introducing faults we can observe how the system responds and compare those results with our theoretical predictions.

One example we observed involved forcing the VBLANK signal low. Instead of disabling the display entirely, the image remained mostly intact but red artifacts appeared along the lower edge of the frame.

This suggests that VBLANK primarily controls frame boundary blanking rather than gating all video output

Tab 9